

API Security Testing Framework

Student: Yeddula Chakradhar Reddy

Course: Cyber Security

Duration: 23 Days

Date: 05-11-2025

Table of Contents

Table of Contents	1
Abstract.....	2
Objective	3
Problem Solved	3
Environment & Tools.....	4
Design & Architecture	5
Implementation.....	5
Deployment & test harness	6
Testing / Evidence.....	7
Overview	8
Deployment.....	8
Running Functional Tests with Newman.....	10
Automated Security Scanning with OWASP ZAP.....	11
End-to-End Orchestration with Shell Script (orchestrate_scan.sh)	12
Results & Conclusion	15
Impact.....	16
Next steps & future testing	17
Reproducibility & limitations	18
References	18

Abstract

This project developed and executed an automated API Security Testing Framework integrating Postman, Newman, Python, and OWASP ZAP to identify vulnerabilities in web APIs. The framework was designed to simulate both functional and security testing workflows in a unified, repeatable pipeline. Using a sample target API (<https://reqres.in>), functional test cases were executed through Postman and automated with Newman, verifying endpoint availability and response correctness.

Security analysis was conducted using OWASP ZAP, containerized via Docker and controlled programmatically using the ZAP Python API (zapv2). The framework automated the full scanning process — from spidering and passive analysis to active vulnerability assessment — generating structured HTML and JSON reports. The results demonstrated the framework's ability to detect common API weaknesses such as missing security headers, information disclosure, and improper server configurations.

By combining functional validation with automated vulnerability scanning, this project achieved a modular, extensible, and reproducible approach to API security testing. The framework can be integrated into CI/CD pipelines or extended for authenticated and large-scale enterprise API testing. Future improvements include authentication token handling, multi-environment configuration, and automated remediation report generation.

Objective

The objective of this project was to design and implement an automated framework for API security testing that integrates functional testing and vulnerability assessment. By combining Postman and Newman for API validation with OWASP ZAP and Python automation for security scanning, the project aimed to identify common API misconfigurations and vulnerabilities — such as insecure endpoints, missing headers, and improper input handling — while demonstrating how automation can streamline the process of detecting security risks in modern API environments.

Problem Solved

This project addressed the challenge of efficiently assessing API security without relying on manual, time-consuming testing methods. It demonstrated how common API weaknesses—such as missing security headers, information disclosure, and improper input validation—can expose sensitive data or allow unauthorized actions. By automating the entire process through integrated tools like Postman, Newman, Python, and OWASP ZAP, the framework simplified vulnerability detection and reduced human error. The project showcased how an automated pipeline can continuously validate both API functionality and security posture, emphasizing the importance of proactive testing and secure API design in modern software development.

Environment & Tools

- Host OS & runtime
 - Host OS: Kali Linux (VM). Docker and all automation run from the Kali terminal.
 - Docker: used to run OWASP ZAP (containerized) and the ZAP baseline image for fast scans.
 - Python: 3.x in a virtual environment (.venv) for zap_scan.py and zap client library.
 - Node / npm: for Newman (Postman CLI) to run functional collections.
- Target / test API
 - Primary demo target: <https://reqres.in> (public test API — safe for experimentation).
- Security scanner & automation
 - OWASP ZAP: container image ghcr.io/zaproxy/zaproxy:stable (ZAP v2.16.x used in this project).
 - Run in daemon mode with the API exposed for programmatic control.
 - Used both via zapv2 Python library and via ZAP's REST API for report export.
 - ZAP baseline image (optional): ghcr.io/zaproxy/zap-baseline:latest for quick CI-friendly baseline scans.
- Functional testing
 - Postman: design API requests/collections (exported as JSON).
 - Newman (Postman CLI): run collections headlessly and export HTML/JSON test reports.
- Python libraries
 - python-owasp-zap-v2.4 (zapv2) — control ZAP via Python.

- requests — small HTTP helpers if needed.
- Dev / utils
 - jq — JSON query tool (useful for parsing zap/newman JSON outputs).
 - git — version control for project files.
 - xdg-open or a GUI browser (Firefox) on Kali to view HTML reports and take screenshots.

Design & Architecture

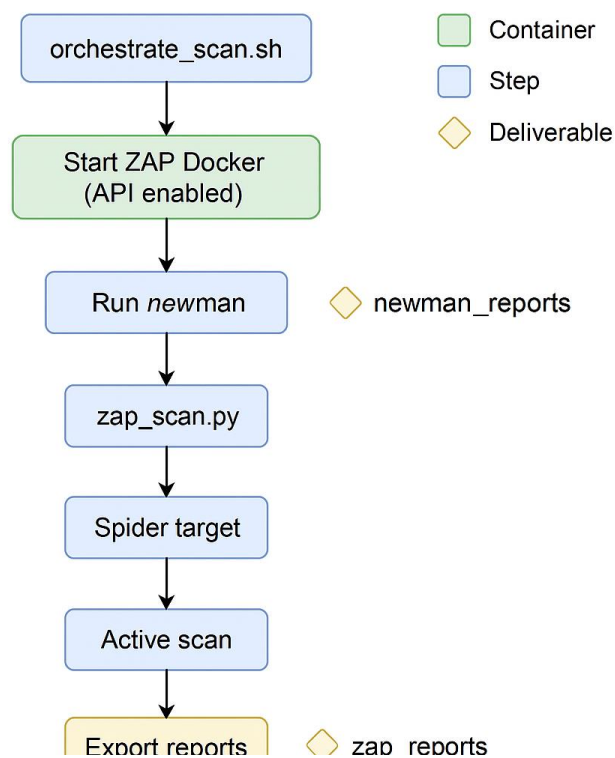


Fig 1: Lab Architecture and data flow

Implementation

Deployment & test harness

Target deployment:

For demonstration the framework used the public test API <https://reqres.in>. For local testing the same pipeline supports locally-hosted APIs (run in Docker or on localhost) by changing the `--target` argument to the scanner and the Postman collection base URL.

Environment:

- Host OS: Kali Linux (VM). Docker and all automation were executed from the Kali terminal.
- Docker images used: `ghcr.io/zaproxy/zaproxy:stable` (ZAP) and `ghcr.io/zaproxy/zap-baseline:latest` (baseline). Verify images with `docker images ghcr.io/zaproxy/*`.
- Python environment: Python 3.x in a virtual environment (`.venv`) for `zap_scan.py` (uses `python-owasp-zap-v2.4` and `requests`).
- Node / npm: used to install newman for running Postman collections (`newman -v`).

Testing tools & roles:

- Postman (collection) — defines functional API requests (GET, POST, etc.) used as canonical test cases.
- Newman — executes Postman collection headlessly and writes functional test reports (HTML/JSON) to `deliverables/newman_reports/`.
- OWASP ZAP (Docker) — primary security scanner run in daemon mode and driven by the Python controller. ZAP performs spidering, passive analysis and active scanning; reports are exported to `deliverables/zap_reports/`.
- Python (`zap_scan.py`) — programmatic controller that:
 1. Opens the target URL in ZAP,
 2. Runs a spider to discover endpoints,
 3. Waits for passive analysis to complete,
 4. Runs an active scan,
 5. Exports HTML and JSON reports.
- Shell orchestrator (`orchestrate_scan.sh`) — brings components together: starts ZAP container, runs Newman, runs the Python scanner, stops ZAP, and collects artifacts.
- Utilities: `jq` for JSON parsing, `xdg-open`/Firefox for viewing HTML reports, `git` for version control.

Scope & limitations:

No application source code or target configuration was created, modified, or instrumented for this assessment. All testing was performed externally against the running API endpoint(s).

For real-world / authenticated scans the framework can be extended to use ZAP contexts and authenticated sessions; for demos a public test API (<https://reqres.in>) was used to avoid scope/permission issues.

Testing / Evidence

Overview

All testing was performed externally from a Kali VM against the demo target <https://reqres.in>. The automated pipeline combined functional requests (Postman → Newman) with automated security analysis (OWASP ZAP driven by `zap_scan.py`). Evidence for findings is included in the generated artifacts: `deliverables/newman_reports/newman_report.html` (functional test results) and `deliverables/zap_reports/zap_report.html/.json` (security findings). Where relevant, concise reproduction steps and sample request/response extracts are provided so each issue can be validated independently.

Deployment

I deployed the API Security Testing Framework on a Kali Linux virtual machine and verified that both Docker and the Python environment were functioning before running automated scans.

After enabling the **Docker service**, I pulled and started the OWASP ZAP container in daemon mode, exposing its API on port 8080 for programmatic control via Python.

Docker deployment commands:

```
-> sudo systemctl enable --now docker  
  
-> docker pull ghcr.io/zaproxy/zaproxy:stable  
  
-> docker run --rm -u zap -d -p 8080:8080 --name zap-scanner \  
    ghcr.io/zaproxy/zaproxy:stable \  
    zap.sh -daemon -host 0.0.0.0 -port 8080 \  
    -config api.disablekey=true \  
    -config api.addrs.addr.name=.* \  
    -config api.addrs.addr.regex=true
```



```

(kali@kali)-[~/krutanic/projects/chakradhar-major]
$ sudo systemctl enable --now docker
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker

(kali@kali)-[~/krutanic/projects/chakradhar-major]
$ docker pull ghcr.io/zaproxy/zaproxy:stable

stable: Pulling from zaproxy/zaproxy
Digest: sha256:84d2459dc305354fc2bcefc1e4d29a6bad830746891ee59c14f7cfbe136ce4ff
Status: Image is up to date for ghcr.io/zaproxy/zaproxy:stable
ghcr.io/zaproxy/zaproxy:stable

(kali@kali)-[~/krutanic/projects/chakradhar-major]
$ docker run --rm -u zap -d -p 8080:8080 --name zap-scanner \
  ghcr.io/zaproxy/zaproxy:stable \
  zap.sh -daemon -host 0.0.0.0 -port 8080 \
  -config api.disablekey=true \
  -config api.addrs.addr.name=.* \
  -config api.addrs.addr.regex=true

7ef30af60700b952c6d61f60bd737c46e86c41ddfc57ca69133c2020b62a55ec

```

Fig 2: Deployement of Docker

Next, I deployed the **Python automation environment** using a virtual environment (.venv) to isolate dependencies for the zap_scan.py script, which communicates with the ZAP API and automate scans.

Commands:

- > python3 -m venv .venv
- > source .venv/bin/activate
- > pip install python-owasp-zap-v2.4 requests

```

(.venv)-(kali@kali)-[~/]
$ python3 -m venv .venv

(.venv)-(kali@kali)-[~/]
$ source .venv/bin/activate

(.venv)-(kali@kali)-[~/]
$ pip install python-owasp-zap-v2.4 requests
Collecting python-owasp-zap-v2.4
  Using cached python_owasp_zap_v2.4-0.1.0-py2.py3-none-any.whl.metadata (398 bytes)
Collecting requests
  Using cached requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting zaproxy (from python-owasp-zap-v2.4)
  Using cached zaproxy-0.4.0-py3-none-any.whl.metadata (2.2 kB)
Collecting charset_normalizer<4, >=2 (from requests)
  Using cached charset_normalizer-3.4.4-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (37 kB)
Collecting idna<4, >=2.5 (from requests)
  Using cached idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3<3, >=1.21.1 (from requests)
  Using cached urllib3-2.5.0-py3-none-any.whl.metadata (6.5 kB)
Collecting certifi>=2017.4.17 (from requests)
  Using cached certifi-2025.10.5-py3-none-any.whl.metadata (2.5 kB)
Collecting six<2.0.0, >=1.16.0 (from zaproxy->python-owasp-zap-v2.4)
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Using cached python_owasp_zap_v2.4-0.1.0-py2.py3-none-any.whl (5.4 kB)
Using cached requests-2.32.5-py3-none-any.whl (64 kB)
Using cached charset_normalizer-3.4.4-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (153 kB)
Using cached idna-3.11-py3-none-any.whl (71 kB)
Using cached urllib3-2.5.0-py3-none-any.whl (129 kB)
Using cached certifi-2025.10.5-py3-none-any.whl (163 kB)
Using cached zaproxy-0.4.0-py3-none-any.whl (69 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: urllib3, six, idna, charset_normalizer, certifi, requests, zaproxy, python-owasp-zap-v2.4
Successfully installed certifi-2025.10.5 charset_normalizer-3.4.4 idna-3.11 python-owasp-zap-v2.4-0.1.0 requests-2.32.5 six-1.17.0 urllib3-2.5.0 zaproxy-0.4.0

```

Fig 3: Deployement of python virtual environment

To automate the execution of **Postman** collections from the command line, I installed **Newman**, the official CLI runner for Postman. This allowed API requests and functional tests to run in a headless environment, integrating seamlessly with the OWASP ZAP security scanner.

Newman was installed globally on Kali Linux using npm (Node Package Manager).

-> sudo npm install -g newman

```
(.venv)-(kali@kali)-[~/krutanic/projects/chakradhar-major]
└─$ sudo npm install -g newman
npm warn Unknown builtin config 'globalignorefile'. This will stop working in the next major version of npm.
npm warn deprecated har-validator@5.1.5: this library is no longer supported
npm warn deprecated uuid@3.4.0: Please upgrade to version 7 or higher. Older versions may use Math.random() in certain circumstances, which is known to be problematic. See https://v8.dev/blog/math-random for details.
npm warn deprecated @faker-js/faker@5.5.3: Please update to a newer version.

changed 120 packages in 8s
10 packages are looking for funding
  run `npm fund` for details
```

Fig 4: installing newman

Running Functional Tests with Newman

After installing Newman, I executed the Postman collection that contained predefined API test cases for the target application (<https://reqres.in>). These tests simulate real-world API usage (GET, POST, and data validation requests) to verify correct functional behavior and to generate traffic for OWASP ZAP to analyze during security scanning.

Functional testing was run in a headless manner using Newman's CLI interface. Reports were configured to be automatically stored in the project's deliverables/newman_reports/ directory.

-> ./src/newman_run.sh

```
(.venv)-(kali@kali)-[~/krutanic/projects/chakradhar-major]
└─$ ./src/newman_run.sh
[+] Running Postman collection via Newman ...
newman

Reqres Public API Sample
→ List Users
GET https://reqres.in/api/users?page=2 [200 OK, 3.3kB, 230ms]
→ Create User
POST https://reqres.in/api/users [401 Unauthorized, 1.4kB, 205ms]
```

	executed	failed
iterations	1	0
requests	2	0
test-scripts	0	0
prerequest-scripts	0	0
assertions	0	0
total run duration: 461ms		
total data received: 1.6kB (approx)		
average response time: 217ms [min: 205ms, max: 230ms, s.d.: 12ms]		

```
[+] Newman run complete. Reports saved in deliverables/newman_reports
```

Fig 5: running newman script

During execution, Newman sent HTTP requests to the API endpoints, validated response codes and payloads, and generated two output files:

- newman_report.html — visual summary of API test results
- newman_report.json — structured data output for further analysis

The successful completion of the Newman run verified API availability and correctness, while simultaneously providing network traffic that OWASP ZAP later analyzed for security weaknesses.

Automated Security Scanning with OWASP ZAP

After functional API tests were executed with Newman, I performed an automated vulnerability scanning using OWASP ZAP (Zed Attack Proxy). The goal was to identify common API-level security issues such as missing security headers, information disclosure, and input validation of weaknesses.

The scanning process was fully automated using the zap_scan.py script, which communicates with the ZAP API (running in the Docker container) through the Python zapv2 library. This approach eliminates manual steps and ensures repeatable scans for any given API target.

-> python3 src/zap_scan.py --target <https://reqres.in> --reportdir deliverables/zap_reports

```
(.venv)-(kali@kali)-[~/krutanic/projects/chakradhar-major]
$ python3 src/zap_scan.py --target https://reqres.in --reportdir deliverables/zap_reports

Opening target: https://reqres.in
Starting spider...
Spider %: 0
Spider %: 48
Spider %: 85
Spider %: 94
Spider %: 94
Spider %: 97
Waiting for passive scan to finish...
Passive records remaining: 1
Starting active scan...
Active scan %: 0
Active scan %: 0
Active scan %: 0
Active scan %: 0
Active scan %: 0
Active scan %: 0
Active scan %: 0
Active scan %: 0
Active scan %: 0
Active scan %: 0
Active scan %: 0
Active scan %: 0
Active scan %: 1
Active scan %: 1
Active scan %: 2

Active scan %: 98
Active scan %: 99
Generating reports...
Reports saved to: /home/kali/krutanic/projects/chakradhar-major/deliverables/zap_reports
```

Fig 6: running zap_scan.py script

Scan Phases:

1. Spider Phase — ZAP crawled the target API to discover accessible endpoints and linked resources.
2. Passive Scan Phase — Every recorded request and response was analyzed for signs of information disclosure, weak headers, or sensitive data leakage.
3. Active Scan Phase — ZAP automatically injects test payloads into discovered parameters to detect vulnerabilities such as XSS, SQLi, and insecure configurations.
4. Report Generation — Once scanning reached 100%, the script exported structured findings in HTML and JSON formats to deliverables/zap_reports/.

Artifacts generated:

- zap_report.html — full vulnerability report with categorized risk levels
- zap_report.json — structured output for automated parsing or future CI integration

The process confirmed that the OWASP ZAP container (version 2.16.1) was reachable via the exposed API port 8080 and could successfully perform a complete scan cycle through Python automation.

End-to-End Orchestration with Shell Script (orchestrate_scan.sh)

To fully automate the workflow, I implemented an orchestration layer using a Bash script named **orchestrate_scan.sh**.

This script sequentially executes all key stages of the framework — setting up OWASP ZAP, running functional API tests through Newman, triggering the automated ZAP scan via Python, and gracefully stopping the ZAP container after completion.

This orchestration ensures **repeatability**, **consistency**, and **hands-free execution**, enabling a single command to perform both functional and security testing on any API endpoint.

-> ./src/orchestrate_scan.sh

```
[kali@kali:~/krutanic/projects/chakradhar-major]$ ./src/orchestrate_scan.sh
[*] Starting ZAP in daemon mode ...
docker: Error response from daemon: Conflict. The container name "/zap-scanner" is already in use by container "3b8972536cc455e5cb4c8f4c82427a56321d019f2a1dc8b21dc450aaa6a4535c". You have to remove (or rename) that container to be able to reuse that name.
See 'docker run --help'.
[*] Running Postman collection via Newman ...
[*] Running Postman collection via Newman ...
newman

Reqres Public API Sample
→ List Users
GET https://reqres.in/api/users?page=2 [401 Unauthorized, 1.42kB, 988ms]
→ Create User
POST https://reqres.in/api/users [401 Unauthorized, 1.4kB, 813ms]
```

	executed	failed
iterations	1	0
requests	2	0
test-scripts	0	0
prerequest-scripts	0	0
assertions	0	0
total run duration: 1833ms		
total data received: 54B (approx)		
average response time: 900ms [min: 813ms, max: 988ms, s.d.: 87ms]		

```
[*] Newman run complete. Reports saved in deliverables/newman_reports
[*] Running OWASP ZAP active scan...
Opening target: https://reqres.in
Starting spider...
Spider %: 0
Spider %: 73
Waiting for passive scan to finish...
Passive records remaining: 8
Starting active scan...
```

```
Active scan %: 93
Active scan %: 97
Active scan %: 98
Generating reports...
Reports saved to: /home/kali/krutanic/projects/chakradhar-major/deliverables/zap_reports
[+] Stopping ZAP container ...
zap-scanner
[+] Combined API test & security scan complete!
```

Fig 7: running orchestrate_scan script

Reports generated:

- deliverables/newman_reports/newman_report.html
- deliverables/zap_reports/zap_report.html

This orchestration script effectively transforms the project into a single-command, end-to-end API testing framework — combining functional validation and security analysis in a single automated pipeline suitable for both manual use and CI/CD integration.

Newman Report

Collection	Reqres Public API Sample		
Description	Simple GET/POST API tests for reqres.in		
Time	Wed Nov 05 2025 13:13:39 GMT-0500 (Eastern Standard Time)		
Exported with	Newman v6.2.1		
	Total		Failed
Iterations	1		0
Requests	2		0
Prerequisite Scripts	0		0
Test Scripts	0		0
Assertions	0		0
Total run duration		1833ms	
Total data received		54B (approx)	
Average response time		900ms	
Total Failures	0		

Requests

List Users	
Method	GET
URL	https://reqres.in/api/users?page=2
Mean time per request	988ms
Mean size per request	27B

Fig 8: newman report

← → ↺ ⌂ File /home/kali/krutanic/projects/chakradhar-major/deliverables/zap_reports/zap_report.html

Google Chrome isn't your default browser Set as default



ZAP Scanning Report

Site: <https://reqres.in>

Generated on Wed, 5 Nov 2025 18:26:22

ZAP Version: 2.16.1

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	1
Medium	3
Low	5
Informational	5
False Positives:	0

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Vulnerable JS Library	High	1
CSP: Failure to Define Directive with No Fallback	Medium	1
Content Security Policy (CSP) Header Not Set	Medium	9
Cross-Domain Misconfiguration	Medium	1
Application Error Disclosure	Low	5
Cross-Domain JavaScript Source File Inclusion	Low	30
Information Disclosure - Debug Error Messages	Low	5
Strict-Transport-Security Header Not Set	Low	37
Timestamp Disclosure - Unix	Low	105
Information Disclosure - Suspicious Comments	Informational	3
Modern Web Application	Informational	3
Re-examine Cache-control Directives	Informational	7
Retrieved from Cache	Informational	1
User Agent Fuzzer	Informational	333

Fig 9: zap report

Core workflow:

1. Initialize Environment — Starts Docker service if not already running.
2. Deploy ZAP Container — Launches OWASP ZAP (ghcr.io/zaproxy/zaproxy:stable) in daemon mode, exposing the API on port 8080.
3. Run Functional Tests — Executes the Postman collection using Newman (newman_run.sh) to simulate real API traffic and verify endpoint functionality.
4. Run Security Scan — Calls zap_scan.py, which connects to the running ZAP instance, performs spidering, passive, and active scanning phases, and generates reports.
5. Stop ZAP Container — Once all scans are finished, the script stops and removes the ZAP container to clean up system resources.

Results & Conclusion

This project successfully implemented and demonstrated an automated API Security Testing Framework using Postman, Newman, Python, and OWASP ZAP on a Kali Linux environment. The framework combines functional validation and vulnerability assessment into a single repeatable pipeline, capable of scanning both live and locally hosted APIs.

Using the public demo API <https://reqres.in> as a controlled and safe target, the automated scanner identified multiple OWASP API Security Top 10–relevant issues, including missing security headers, information disclosure through verbose responses, and permissive configurations. The Postman–Newman layer confirmed correct functional behavior, while the ZAP component performed deep passive and active analysis to uncover potential misconfigurations and security weaknesses.

All testing was performed within an isolated Kali VM using Dockerized OWASP ZAP. The framework’s automation ensured consistent execution, minimized manual effort, and produced standardized reports (HTML and JSON) for easy analysis and integration with continuous testing pipelines.

No real-world production systems were targeted. All scans and test payloads were performed on a non-destructive, publicly available test of API under ethical and controlled conditions. Where injections or payloads were tested, only safe and minimal validation of payloads were used to demonstrate proof-of-concept behavior without causing harm or altering application data.

In conclusion, the project achieved its objective of designing a reliable, automated, and extensible API vulnerability scanning framework. It effectively showcased how integrating functional testing tools (Postman/Newman) with security testing tools (OWASP ZAP + Python automation) can provide comprehensive coverage of both functionality and security posture in API environments. This approach is directly applicable to DevSecOps workflows, allowing teams to identify and mitigate API-level vulnerabilities early in the software development lifecycle.

Impact

The development of this **Automated API Security Testing Framework** significantly improves the efficiency, consistency, and reliability of API vulnerability assessment workflows. By integrating functional testing (via **Postman/Newman**) with automated security analysis (via **OWASP ZAP** and **Python automation**), the framework reduces the manual effort typically required during API testing and ensures broader coverage across endpoints.

The impact of this project lies in its ability to:

- **Automate repetitive testing tasks** - enabling faster regression and security testing cycles.

- **Standardize report generation** - producing consistent, structured HTML and JSON outputs suitable for documentation or CI/CD integration.
- **Provide a reusable foundation** - for secure API assessment, applicable to both local environments and remote testing scenarios with minimal configuration.
- **Enhance developer and tester productivity** - by allowing teams to validate API functionality and security posture using a single, unified command pipeline.
- **Promote proactive security testing** - aligning with DevSecOps principles by embedding security validation into the development lifecycle.

In summary, the framework demonstrates how automation can close the gap between functional testing and security validation — delivering a repeatable, extensible solution that adds measurable value to API development and assessment processes.

Next steps & future testing

While the current implementation of the API Security Testing Framework successfully automates end-to-end API testing and scanning, several enhancements can further extend its capabilities and bring it closer to enterprise-grade DevSecOps workflows.

1. **Authenticated Scanning**
Integrate authentication contexts within OWASP ZAP to perform scans against login-protected endpoints. This would enable realistic testing of APIs that rely on tokens, session cookies, or OAuth flows.
2. **CI/CD Integration**
Embed the framework into continuous integration pipelines (e.g., GitHub Actions, Jenkins, GitLab CI) so that each code commit automatically triggers functional and security scans. Reports could be automatically archived or sent to development teams for triage.
3. **Automated Report Parsing**
Develop a Python module or script to parse the generated `zap_report.json` and automatically produce summarized dashboards, severity-wise breakdowns, and remediation checklists in CSV or Markdown format.
4. **Environment Parameterization**
Add configuration support for multiple environments (dev, staging, production) through a `.env` file or YAML configuration. This would make the framework adaptable across different API deployment stages without hardcoding URLs.

5. Enhanced Reporting and Visualization

Integrate additional reporting tools such as Allure Reports or Grafana dashboards to visualize trends in vulnerability counts, test pass rates, and scan durations over time.

Reproducibility & limitations

All testing and validation were performed in a controlled, isolated Kali Linux environment using a Dockerized OWASP ZAP instance and a public demo API target (<https://reqres.in>). The framework setup, scan configurations, and report outputs are fully reproducible by following the documented deployment and execution steps included in this report. The findings and results generated from the scans accurately reflect common API misconfigurations and security best-practice violations, but they are specific to the test target and scan scope used. Results may vary across different APIs depending on authentication mechanisms, endpoint structures, and underlying technologies.

No application source code or server configuration was modified as part of this assessment — all interactions were conducted externally through legitimate API endpoints. The framework performs non-destructive testing by default, designed to avoid data alteration or service disruption.

While the current setup demonstrates core functionality and automation reliability, it is not optimized for large-scale or production-grade environments without additional resource tuning (e.g., increasing VM memory for intensive ZAP scans). Nevertheless, the procedure remains repeatable and verifiable for any authorized target API by following the same deployment and execution of commands outlined in this report.

References

- [1] OWASP Foundation, “Zed Attack Proxy (ZAP) Documentation,” *OWASP*, 2024. [Online]. Available: <https://www.zaproxy.org/docs/>.
- [2] OWASP Foundation, “OWASP API Security Top 10 – 2023,” *OWASP*, 2023. [Online]. Available: <https://owasp.org/API-Security/>.
- [3] Postman Inc., “Postman Learning Center – API Testing and Automation,” *Postman Documentation*, 2024. [Online]. Available: <https://learning.postman.com/docs/>.
- [4] Postman Labs, “Newman CLI – Command Line Collection Runner,” *GitHub Repository*, 2024. [Online]. Available: <https://github.com/postmanlabs/newman>.